Algoritmos y Estructura de datos III — Año 2026 – 2do. Periodo

Prof. Cristian Cappo (TQ) / Prof. Luis Moré (TR) / Prof. Gustavo Galeano (TS)

Trabajo Práctico 3 – U4 (Árboles B+ persistentes en disco)

Fecha de publicación: sábado 03/10/2026
Fecha Entrega Límite: lunes 12/10/2026 23:59 hs.
Defensa oral: miércoles 14 y jueves 15/10/2026 (Semana 11, en clase)
TP = 100 pts (Ejercicio 1: 50p · Ejercicio 2: 25p · Defensa oral: 25p)

Cada archivo fuente deberá tener la identificación del grupo (g##) y los integrantes del grupo con la indicación del número de tarea. Por cada integrante colocar: apellido, nombre, CIC y sección.

Debe incluir la Declaración de Honor con el texto mostrado en la primera clase.

La defensa oral es individual y es requisito para todo el TP

Cualquier integrante puede ser interrogado sobre cualquier parte del código. Un integrante que no pueda explicar el código que su grupo entregó recibe **0 (cero) en el trabajo práctico completo** —no solo en la componente de defensa—, sin afectar a los demás integrantes. Entregar código que no se puede defender equivale a no entregarlo.

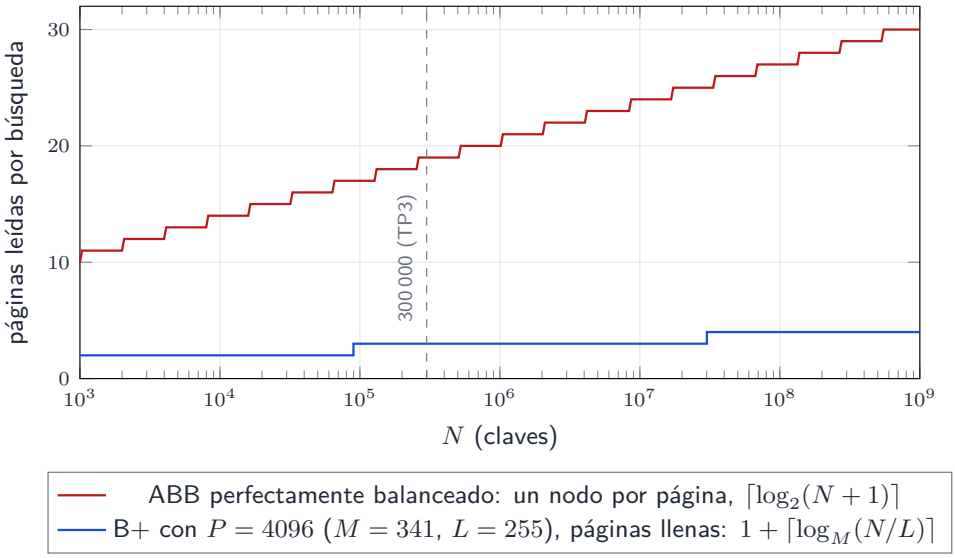
Contexto y motivación

IMDb publica su catálogo como archivos de texto separados por tabuladores. El archivo `title.basics.tsv` tiene un renglón por título —más de once millones— con su identificador `tconst` (`tt0111161`), el tipo, el título, el año, la duración y los géneros. Una plataforma de catálogo atiende dos consultas típicas:

1. *«Traeme la ficha de `tt0111161`»*: acceso por clave exacta.
2. *«Todos los títulos con identificador entre `tt0100000` y `tt0100999`»*: acceso por rango.

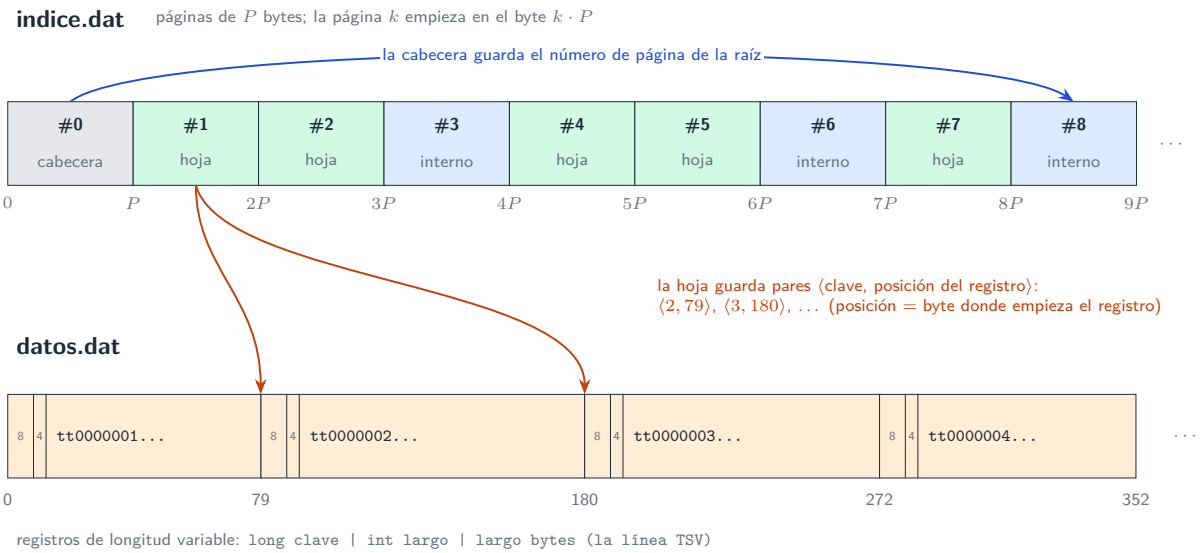
Mientras los datos caben en memoria, un ABB balanceado como el del TP2 resuelve las dos. Pero un sistema de bases de datos no puede suponer eso: los datos viven en disco, y el disco no se lee por bytes sino por **páginas** (bloques de, típicamente, 4096 bytes). Leer una página cuesta del orden de 10^{-4} s en un SSD y 10^{-2} s en un disco magnético; comparar dos enteros en memoria, del orden de 10^{-9} s. Cuando los datos están en disco, **la unidad de costo es la página leída**, no la comparación.

Con ese costo, un ABB es una mala idea aunque esté perfectamente balanceado: cada nodo guarda una clave, y bajar un nivel es leer otra página. Con $N = 300\,000$ claves son 19 lecturas por búsqueda. Un **árbol B+** guarda en cada nodo una página entera —cientos de claves— y su altura crece como $\log_M N$ con M en los cientos: con páginas de 4096 bytes, las mismas 300 000 claves se encuentran leyendo **3 páginas**.



En el TP2 contaban *visitas a nodos* para mostrar que `kEsimo` no recorría el árbol. Acá un nodo es una página del archivo y cada visita es un **acceso a disco**. Van a contar **páginas leídas y escritas**, y con esos números van a mostrar por qué todo motor de bases de datos indexa con árboles B+.

El trabajo tiene dos partes encadenadas. El **Ejercicio 1** es el árbol B+ en sí: un índice `clave` → `posición` que vive en el archivo `indice.dat`, página por página, y que se puede cerrar y volver a abrir. El **Ejercicio 2** lo usa como índice de un archivo de datos real construido con IMDb y lo mide. El Ejercicio 1 concentra el puntaje; conviene terminarlo y probarlo con la página de juguete ($P = 60$) antes de tocar el dataset.



Cómo encarar el trabajo

Tienen 9 días corridos: el plazo es corto y no admite empezar tarde. Una secuencia razonable:

Días	Objetivo
1	Leer el formato de página. Deducir en papel L y M para $P = 60$ y $P = 4096$. Dibujar a mano la traza de la sección «Ejemplo de traza» **antes** de escribir código, con los números de página.
2	Cabecera, `crear`/`abrir`/`close`, lectura y escritura de una página. `buscar` sobre un árbol de un solo nivel.
3–4	`insertar`: split de hoja, split de interno, propagación y nueva raíz. Reproducir la traza fila por fila, incluidas las columnas de lecturas y escrituras.
5	`buscarRango` por la cadena de hojas, `verificarInvariantes`, `TestArbolBMas`.
6–7	Ejercicio 2: `ArchivoDatos`, `BMasApp`, construcción con IMDb, reapertura en una ejecución separada.
8–9	`ExperimentoBMas`, README con el análisis y la declaración de uso de IA, y **ensayo de la defensa**: háganse las preguntas entre ustedes, tracen un split en el pizarrón.

Un consejo que vale puntos: si a mitad de camino no pueden explicarle a un compañero por qué en la traza la página nueva de la inserción de 40 es la #5 y no otra, paren de programar y vuelvan al dibujo. El código B+ escrito sin entender qué se copia y qué se mueve en un split funciona con diez claves y se rompe con mil.

Ejercicio 1 (50p) — `ArbolBMas`: árbol B+ en páginas de disco

Implemente la clase `ArbolBMas`, un árbol B+ cuyas claves y posiciones son `long` y cuyos nodos son **páginas de tamaño fijo** del archivo `indice.dat`.

La estructura y su invariante

Sea M el máximo de hijos de un nodo interno y L el máximo de pares ⟨clave, posición⟩ de una hoja. Las propiedades que deben cumplirse **siempre**, al retornar de cualquier operación:

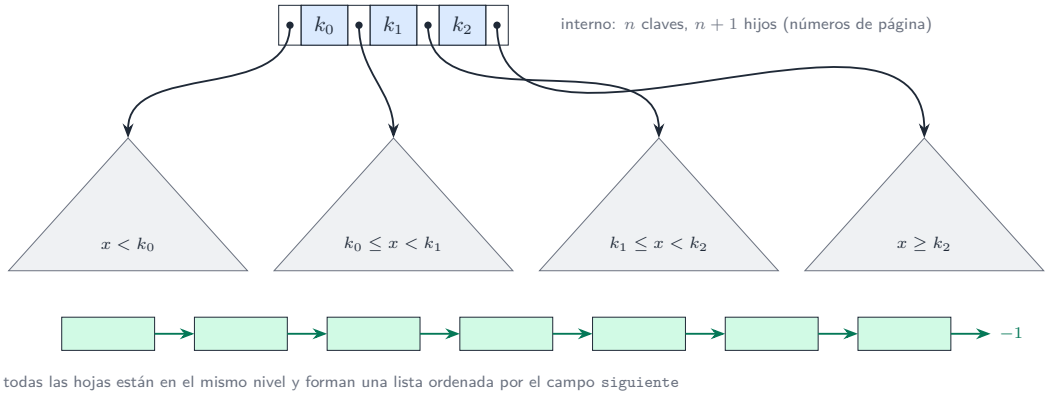
I1. Los datos están solo en las hojas. Cada hoja guarda pares ⟨clave, posición⟩ ordenados por clave, sin claves repetidas. Todas las hojas están en el mismo nivel.

I2. Separadores. Un nodo interno con n claves $k_0 < k_1 < \cdots < k_{n-1}$ tiene $n+1$ hijos $c_0, \ldots, c_n$, y toda clave x del subárbol c_i cumple $k_{i-1} \le x < k_i$ (sin cota a la izquierda para c_0 ni a la derecha para c_n).

I3. Ocupación. Una hoja que no es raíz tiene entre $\lceil L/2 \rceil$ y L pares. Un interno que no es raíz tiene entre $\lceil M/2 \rceil$ y M hijos. Una raíz hoja tiene entre 0 y L pares; una raíz interna, entre 2 y M hijos.

I4. Cadena de hojas. Cada hoja guarda el número de página de la hoja siguiente en orden de claves; la última guarda -1.

I5. Referencias físicas. Un hijo, una hoja siguiente o la raíz se identifican por su **número de página** en `indice.dat`, nunca por una referencia Java.



Regla que define el ejercicio

El árbol vive en el disco. Ninguna operación puede cargar `indice.dat` completo ni mantener el árbol como objetos Java enlazados. Durante una operación pueden estar en memoria la cabecera y, como máximo, las páginas del camino raíz–hoja de esa operación ($O(h)$ páginas). Sin cache (que es el modo por defecto), **cada lectura de nodo va al disco**, y al retornar `insertar` todas las páginas modificadas ya fueron escritas en el archivo. Un árbol que vive en RAM y se serializa al cerrar es otra estructura: **el Ejercicio 1 se califica con 0**, aunque las búsquedas den bien.

Organización física de `indice.dat`

El archivo es una secuencia de páginas de P bytes. La página k empieza en el byte $k \cdot P$. La **página 0** es la cabecera del archivo; los nodos ocupan las páginas $1, 2, \ldots$. Todos los enteros se escriben en **big-endian**, que es lo que hacen `RandomAccessFile`, `DataOutputStream` y `ByteBuffer` por defecto. Los bytes no usados de una página quedan en 0. Cada lectura o escritura de un nodo transfiere la página completa (P bytes): **una E/S = una página**.

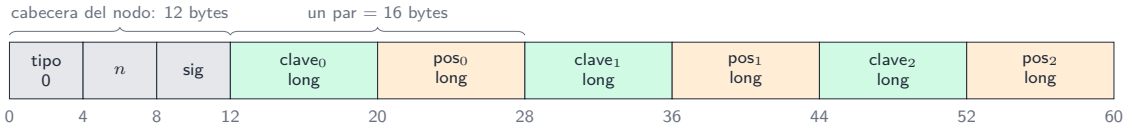
Cabecera (página 0). Ocupa una página entera; los primeros 40 bytes son:

Offset	Tipo	Campo
0	`int`	`MAGIC` = `0x41454433` (los caracteres `AED3`)
4	`int`	versión del formato = 1
8	`int`	tamaño de página P
12	`int`	M (máximo de hijos de un interno)
16	`int`	L (máximo de pares de una hoja)
20	`int`	número de página de la raíz
24	`int`	cantidad de páginas del archivo, **incluida la cabecera**
28	`int`	niveles del árbol (raíz hoja = 1)
32	`long`	cantidad de claves

Página de nodo. Los primeros 12 bytes son comunes: `int tipo` (0 = hoja, 1 = interno), `int n` (cantidad de claves) y `int siguiente` (en una hoja, el número de la hoja siguiente o -1; en un interno, siempre -1).

- **Hoja:** a partir del byte 12, los n pares uno detrás de otro: `long clave` en $12 + 16i$ y `long posicion` en $12 + 16i + 8$.
- **Interno:** los hijos `int` en $12 + 4i$ ($i = 0..n$) y las claves `long` en $12 + 4M + 8i$ ($i = 0..n-1$). El bloque de hijos tiene siempre lugar para M; las claves empiezan después, aunque el nodo no esté lleno.

Página hoja ($P = 60 \Rightarrow L = 3$)



Página interna ($P = 60 \Rightarrow M = 4$)



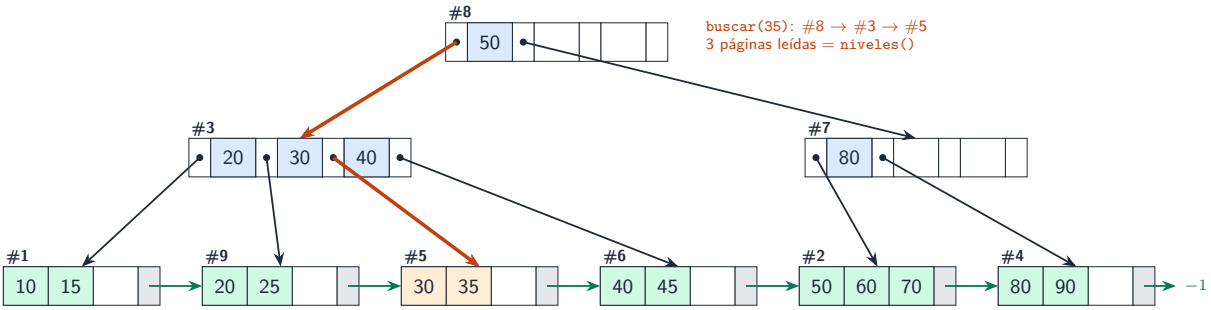
M y L no se eligen: se calculan

L y M son el mayor entero que hace caber una hoja y un interno en P bytes con este formato. Con $P = 60$ dan $L = 3$ y $M = 4$, como en la figura: verifíquenlo. Ustedes deducen la fórmula general, la programan en el constructor a partir de P (**no** como constantes) y en el README justifican los valores para $P = 4096$, cuántos bytes quedan sin usar y por qué $M \neq L$. Es la primera pregunta de la defensa.

Cómo funciona cada operación

Búsqueda

Se lee la raíz. En un interno se baja por el hijo c_i con $i =$ *cantidad de claves del nodo que son* $\leq x$ (búsqueda binaria dentro de la página). En la hoja se busca x. Se lee **exactamente una página por nivel**: `buscar` lee `niveles()` páginas, exista o no la clave.



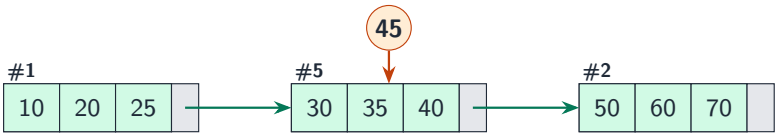
En #8: $35 < 50$, hijo 0. En #3: $30 \leq 35 < 40$, hijo 2. En la hoja #5 se encuentra 35 con posición 350.
Regla de descenso: se baja por el hijo $i =$ cantidad de claves del nodo que son $\leq x$.

Inserción

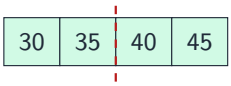
Se baja como en la búsqueda, recordando el camino. Si la clave ya está, no se modifica nada y se retorna `false`. Si no, se inserta en orden en la hoja. Si la hoja desborda, se parte; si el padre desborda al recibir el separador, se parte; y así hacia arriba. Si se parte la raíz, se crea una raíz nueva y el árbol gana un nivel. Las reglas son exactas, porque de ellas dependen los números de página de la traza:

1. **Split de hoja** ($L+1$ pares en memoria): la hoja original conserva los primeros $\lceil (L+1)/2 \rceil$; el resto va a una hoja nueva. El separador que sube es la **primera clave de la hoja nueva, copiada**: sigue estando en la hoja. La hoja nueva hereda el `siguiente` de la original, y la original pasa a apuntar a la nueva.
2. **Split de interno** (M claves y $M+1$ hijos en memoria): el nodo original conserva los primeros $\lceil (M+1)/2 \rceil$ hijos y las claves entre ellos; la clave siguiente **sube y desaparece del nivel**; el resto de claves e hijos va al interno nuevo.
3. **Asignación de páginas:** toda página nueva se agrega al final del archivo, con número = cantidad de páginas actual. En una inserción con varias divisiones se asignan de abajo hacia arriba: primero la hoja nueva, luego cada interno nuevo, al final la raíz nueva.

Antes: `insertar(45)` llega a la hoja #5, que está llena ($L = 3$)

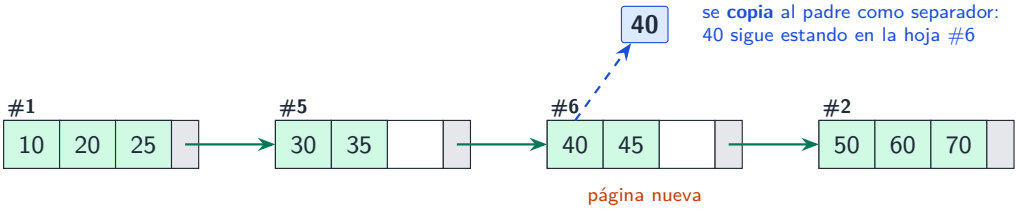


en memoria, la hoja tendría $L + 1 = 4$ pares:



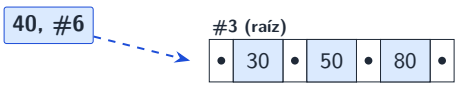
izquierda conserva $\lceil (L + 1)/2 \rceil = 2$

Después: la mitad derecha va a una página nueva, la #6, al final del archivo

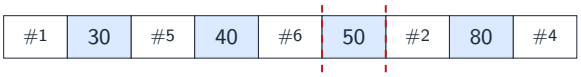


#6.siguiente ← #5.siguiente (= #2), luego #5.siguiente ← #6. Si la nueva hoja recibe -1, la cadena se corta en el medio.

1. El separador 40 y el puntero a #6 entran en el padre #3, que ya tenía 3 claves ($M = 4$)

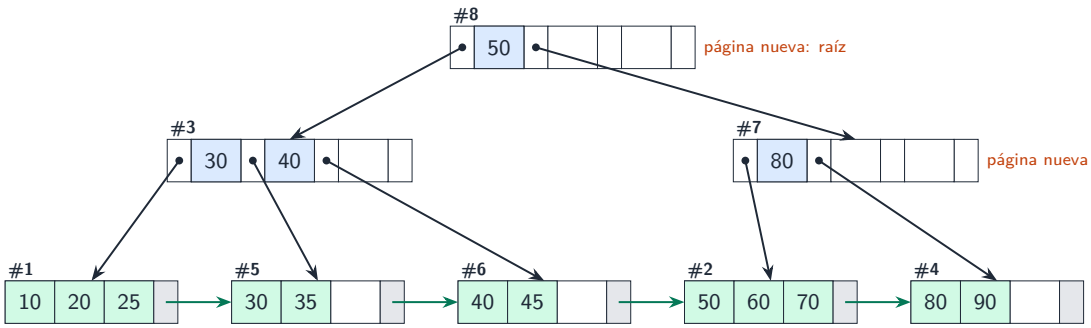


en memoria: 4 claves y 5 hijos



izquierda: $\lceil (M + 1)/2 \rceil = 3$ hijos
la clave del medio (50) **sube**

2. #3 se parte en #3 y #7 (nueva); 50 sube a una raíz nueva #8: el árbol gana un nivel



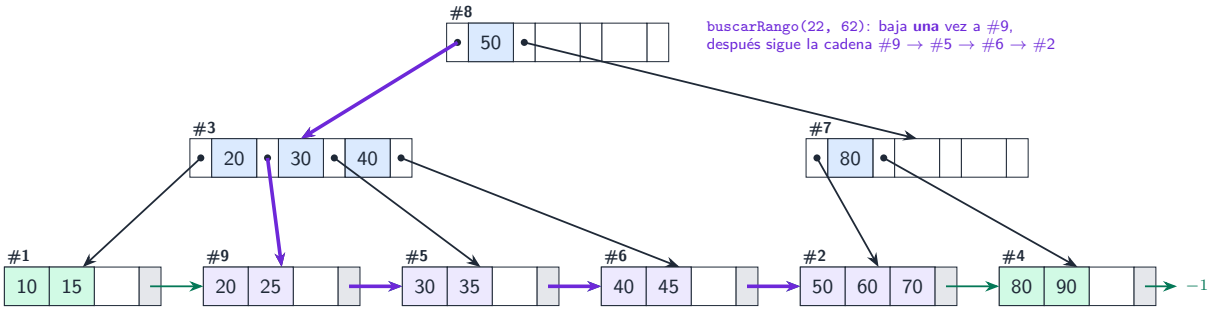
50 ya **no** está en #3 ni en #7: en un interno la clave del medio se mueve, no se copia.
Páginas asignadas en esta inserción, en orden: #6 (hoja), #7 (interno), #8 (raíz). Escrituras: $2s + 1 = 5$ con $s = 2$ divisiones.

Contrato de E/S de `insertar`

Una inserción lee **`niveles()` páginas** (las del camino, una vez cada una) y escribe $2s + 1$ **páginas**, con s = cantidad de divisiones: cada nodo que se parte escribe dos páginas (él y la nueva), y el último nodo que absorbe el separador —o la raíz nueva— escribe una. Un duplicado lee `niveles()` páginas y no escribe ninguna. No se reescriben páginas que no cambiaron. La cabecera se reescribe cuando cambia y **no se cuenta**.

Búsqueda por rango

`buscarRango(desde, hasta, c)` baja **una sola vez** a la hoja donde estaría `desde` y desde ahí sigue la cadena de hojas, entregando al consumidor `c`, en orden, cada par con clave en [desde, hasta]. Regla de parada: termina al encontrar una clave $>$ `hasta`, o al llegar al final de una hoja cuya última clave sea $\geq$ `hasta`, o cuando `siguiente` $= -1$. Es decir, **pasa a la hoja siguiente solo si la última clave de la actual es $<$ `hasta`**. No se aceptan varias búsquedas independientes desde la raíz.



Entrega 25 30 35 40 45 50 60 y se detiene en #2 al ver 70 > 62. Páginas leídas: 3 (descenso) + 3 (hojas siguientes) = 6.
Pasa a la hoja siguiente solo si la última clave de la hoja actual es < hasta: con buscarRango(20, 45) se detiene en #6 sin leer #2.

Métodos

Método	Funcionalidad
`static ArbolBMas crear(String ruta, int tamPagina)`	Crea un índice nuevo (si el archivo existe, lo reemplaza). Árbol vacío: la raíz es una hoja vacía en la página 1, `niveles()` $= 1$, `cantidadPaginas()` $= 2$. Lanza `TamPaginaInvalidoException` si $P < 40$, $L < 2$ o $M < 3$.
`static ArbolBMas abrir(String ruta)`	Abre un índice existente leyendo **solo la cabecera**. Valida `MAGIC`, versión, que M y L correspondan a P, y que el largo del archivo sea `cantidadPaginas`$\cdot P$. Si no, `FormatoInvalidoException`.
`boolean insertar(long clave, long posicion)`	Inserta el par. `false` si la clave ya existía (no modifica nada).
`long buscar(long clave)`	Posición del registro, o -1.
`int[] caminoBusqueda(long clave)`	Números de página leídos de la raíz a la hoja. Lo usan `BMasApp` y la defensa.
`int buscarRango(long desde, long hasta, ConsumidorEntrada c)`	Entrega los pares de [desde, hasta] en orden y retorna cuántos entregó. `RangoInvalidoException` si `desde` $>$ `hasta`.

Método	Funcionalidad
`int niveles()`, `int raiz()`, `int cantidadPaginas()`, `long cantidadClaves()`	Lo que dice la cabecera.
`int tamPagina()`, `int ordenInterno()`, `int capacidadHoja()`	P, M y L.
`long paginasLeidas()`, `long paginasEscritas()`, `void reiniciarContadores()`	Contadores de páginas de **nodo** transferidas desde/hacia el disco. Arrancan en 0 al crear o abrir.
`String volcarPagina(int nro)`	Una página, en el formato de abajo.
`String volcarArbol()`	El árbol, una línea por nivel, en el formato de abajo.
`EstadisticasBMas estadisticas()`	Recorre el árbol: `hojas`, `internos`, `ocupacionHojas` $= \sum n/(\text{hojas} \cdot L)$, `ocupacionInternos` $= \sum(n+1)/(\text{internos} \cdot M)$. Campos públicos.
`String verificarInvariantes()`	`"OK"` o la descripción de la primera violación encontrada.
`void close()`	Escribe la cabecera y cierra. La clase implementa `AutoCloseable`.

`ConsumidorEntrada` es una interface propia con un único método `void aceptar(long clave, long posicion)`; así `buscarRango` no necesita armar ninguna lista y se puede llamar con una lambda.

Diagnóstico sin efectos. `volcarPagina`, `volcarArbol`, `estadisticas` y `verificarInvariantes` leen el disco pero **no modifican los contadores**: son para mirar, no forman parte de la operación que se mide.

Formatos exactos (los usa la corrección; deben coincidir carácter por carácter):

```
volcarPagina(5)  ->  #5 H n=2 sig=6 [30:300 35:350]     // hoja: clave:posicion
volcarPagina(3)  ->  #3 I n=3 [#1 20 #9 30 #5 40 #6]     // interno: hijo clave hijo ...
volcarArbol()    ->  1: #8(50)
                     2: #3(20,30,40) | #7(80)
                     3: #1[10,15] | #9[20,25] | #5[30,35] | #6[40,45] | #2[50,60,70] | #4[80,90]
```

Internos entre paréntesis, hojas entre corchetes, nodos de un mismo nivel de izquierda a derecha separados por `" | "`. El árbol vacío se vuelca como `1: #1[]`.

Verificador de invariantes

`verificarInvariantes()` recorre el árbol desde la raíz y comprueba, como mínimo: que el largo del archivo sea `cantidadPaginas`$\cdot P$; que cada nodo tenga claves estrictamente crecientes y respete las cotas de sus separadores (I2); que todas las hojas estén en el nivel `niveles()` (I1); que las ocupaciones estén en sus rangos (I3); que la cadena de hojas recorra exactamente las hojas del árbol de izquierda a derecha y termine en -1 (I4); que la cantidad de claves coincida con la cabecera y que toda página 1..`cantidadPaginas`-1 sea alcanzable **exactamente una vez**. Lo pueden hacer recursivo con $O(h)$ páginas en memoria: para la cadena basta recordar cuál fue la última hoja visitada.

Ejemplo de traza

Con $P = 60$ ($L = 3$, $M = 4$), partiendo de `crear`, insertar las claves

$$50,\ 20,\ 80,\ 10,\ 30,\ 60,\ 90,\ 40,\ 70,\ 25,\ 35,\ 45,\ 15$$

en ese orden, con posición = 10 · clave. Reinicien los contadores antes de cada fila. R y W son `paginasLeidas()` y `paginasEscritas()` de esa inserción; «págs» es `cantidadPaginas()`. La columna `volcarArbol()` tiene que coincidir carácter por carácter.

Operación	R	W	págs	`volcarArbol()`
(crear)	–	–	2	`1: #1[]`
`insertar(50)`	1	1	2	`1: #1[50]`
`insertar(20)`	1	1	2	`1: #1[20,50]`
`insertar(80)`	1	1	2	`1: #1[20,50,80]`
`insertar(10)`	1	3	4	`1: #3(50) / 2: #1[10,20] | #2[50,80]`
`insertar(30)`	2	1	4	`1: #3(50) / 2: #1[10,20,30] | #2[50,80]`
`insertar(60)`	2	1	4	`1: #3(50) / 2: #1[10,20,30] | #2[50,60,80]`
`insertar(90)`	2	3	5	`1: #3(50,80) / 2: #1[10,20,30] | #2[50,60] | #4[80,90]`
`insertar(40)`	2	3	6	`1: #3(30,50,80) / 2: #1[10,20] | #5[30,40] | #2[50,60] | #4[80,90]`
`insertar(70)`	2	1	6	`2: ... | #2[50,60,70] | ...` (raíz igual)
`insertar(25)`	2	1	6	`2: #1[10,20,25] | ...`
`insertar(35)`	2	1	6	`2: ... | #5[30,35,40] | ...`
`insertar(45)`	2	5	9	ver abajo (3 niveles)
`insertar(15)`	3	3	10	ver abajo

Después de `insertar(45)`:

```
1: #8(50)
2: #3(30,40) | #7(80)
3: #1[10,20,25] | #5[30,35] | #6[40,45] | #2[50,60,70] | #4[80,90]
```

Después de `insertar(15)` (el árbol de las figuras de búsqueda y rango):

```
1: #8(50)
2: #3(20,30,40) | #7(80)
3: #1[10,15] | #9[20,25] | #5[30,35] | #6[40,45] | #2[50,60,70] | #4[80,90]
```

Observen tres momentos. En `insertar(10)` se parte la raíz hoja: aparecen la hoja #2 y la raíz #3. En `insertar(40)` se parte una hoja **del medio**: la nueva #5 queda entre #1 y #2, y la cadena pasa a ser #1→#5→#2→#4. En `insertar(45)` hay dos divisiones y una raíz nueva: se asignan, en este orden, #6, #7 y #8, y se escriben $2 \cdot 2 + 1 = 5$ páginas.

Fase B — consultas sobre el árbol final. Reinicien los contadores antes de cada fila.

Operación	Resultado	Páginas leídas
`buscar(10)` / `buscar(90)`	100 / 900	3 cada una
`caminoBusqueda(35)`	`[8, 3, 5]`	—
`buscar(33)`	−1	3 (una clave inexistente también cuesta un camino completo)
`buscarRango(31, 35, c)`	35	3
`buscarRango(22, 62, c)`	25 30 35 40 45 50 60	6 = 3 + hojas #5, #6, #2
`buscarRango(20, 45, c)`	20 25 30 35 40 45	5: termina en #6 porque su última clave es 45 ≥ 45
`insertar(35, 1)`	`false`	3 leídas, 0 escritas
`buscarRango(9, 5, c)`	`RangoInvalidoException`	—

Fase C — persistencia. `close()` y luego `abrir` sobre el mismo archivo: `raiz()` = 8, `niveles()` = 3, `cantidadPaginas()` = 10, `cantidadClaves()` = 13, el archivo mide $10 \cdot 60 = 600$ bytes; `buscar(45)` = 450 y `buscar(15)` = 150 con 6 páginas leídas entre las dos; `verificarInvariantes()` = `"OK"`.

Hagan esta verificación en papel antes de programar. Reproduzcan a mano la traza con los números de página y las columnas R y W. Es una hora que les ahorra tres días.

Clase de prueba

Cree la clase `TestArbolBMas` que, con $P = 60$, ejecute e imprima los siguientes casos, cada uno con su resultado y `OK`/`FALLA`:

1. árbol inicialmente vacío (niveles, páginas, `buscar` sobre vacío, `volcarArbol`);
2. inserciones sin división (50, 20, 80);
3. división de una hoja: la raíz hoja (10) y una hoja intermedia con su reenlace (40);
4. división de un nodo interno (45);
5. división de la raíz y creación de una raíz nueva (45);
6. crecimiento del árbol a tres niveles;
7. búsqueda de la primera y de la última clave, y `caminoBusqueda`;
8. búsqueda de una clave inexistente;
9. búsqueda por rango dentro de una hoja;
10. búsqueda por rango que atraviesa varias hojas, con la regla de parada;
11. cerrar y volver a abrir el archivo (cabecera, tamaño, archivo inválido);
12. repetir búsquedas después de reabrir.

Imprima además la traza completa (`volcarArbol()`, R, W y páginas después de cada inserción) y `verificarInvariantes()` después de cada una. El caso 13 —construcción con el dataset de 20 MB o más— es el Ejercicio 2.

Sobre las excepciones

- `FormatoInvalidoException`, **chequeada** (extiende `IOException`): `abrir` sobre un archivo que no es un índice válido.
- `RangoInvalidoException`, **no chequeada**: `desde` > `hasta`.
- `TamPaginaInvalidoException`, **no chequeada**: un P que no admite $L \geq 2$ y $M \geq 3$.

Justifiquen en un comentario por qué cada una es de ese tipo, con el criterio de los TP anteriores.

Observaciones

- La solución debe alzarse en el sitio indicado en el aula virtual. Debe incluir comentarios en los archivos fuente.
- Se considerará el orden (espaciado, indentación, comentarios), el estilo (nombres de variables y funciones, comentarios en métodos y clases) y los datos (lo requerido).
- **NO puede usar ninguna estructura de datos del API de Java; debe implementar todas las que necesite (esto invalidará su ejercicio).** Queda prohibido `ArrayList`, `LinkedList`, `Stack`, `ArrayDeque`, `TreeMap`, `TreeSet`, `HashMap`, `HashSet`, `PriorityQueue`, `Arrays.*`, `Collections.*`, `System.arraycopy` y cualquier clase de `java.util.*` salvo `Random`, `Scanner`, `Iterator` e `Iterable`. Se permiten los arreglos nativos (`int[]`, `long[]`, `byte[]`) y las clases de `java.io` / `java.nio` para archivos (`RandomAccessFile`, `FileChannel`, `ByteBuffer`, `Data*Stream`, `Buffered*`).
- Quedan prohibidos la serialización de objetos (`ObjectOutputStream`) y el mapeo del archivo completo en memoria (`FileChannel.map`): las páginas se leen y escriben una por una, explícitamente.

  Por qué esta restricción: el objetivo es entender cómo se representa un árbol en almacenamiento secundario. Un `TreeMap` serializado resuelve las búsquedas y no deja nada para defender.

Ampliaciones opcionales

Pueden compensar hasta 5 puntos perdidos en la entrega, sin que el TP supere los 100. Se evalúan en la defensa.

- **Eliminación** con redistribución entre hermanos, fusión y reducción de altura, manteniendo I1–I5 y `verificarInvariantes() = "OK"`.
- **Buffer de páginas** propio (por ejemplo LRU de k páginas) que se pueda **desactivar**; con buffer, los contadores cuentan solo las páginas que realmente van al disco. Todas las mediciones del Ejercicio 2 se hacen con el buffer desactivado.

Rúbrica — Ejercicio 1

Rúbrica	Puntaje
Diseño físico: M y L calculados desde P, cabecera completa, páginas de tamaño fijo con el formato byte a byte del enunciado	8
`buscar` con exactamente una lectura por nivel, `caminoBusqueda`	6
Split de hoja: separador copiado, reenlace de la cadena (incluida una hoja intermedia), páginas asignadas al final	8
Split de interno: la clave del medio sube, propagación, raíz nueva; contrato de E/S (h lecturas, $2s + 1$ escrituras)	10
`buscarRango` por la cadena de hojas con la regla de parada	6
Persistencia: `crear`/`abrir`/`close`, reapertura sin reconstruir, lectura real del disco, `FormatoInvalidoException`	6
`verificarInvariantes` y `TestArbolBMas` según la especificación (12 casos y traza impresa)	5
Orden, estilo, comentarios y datos	1
Total	**50**

Ejercicio 2 (25p) — Índice sobre IMDb y experimentación

Use el `ArbolBMas` del Ejercicio 1 como índice de un archivo de datos real y mida, con páginas leídas y no con impresiones, por qué conviene.

El dataset

Descarguen `title.basics.tsv.gz` de `https://datasets.imdbws.com/` (uso personal y no comercial según las condiciones de IMDb; **no lo incluyan en la entrega**). Trabajen con las **primeras 300 000 filas de datos** (sin contar el encabezado): el archivo está ordenado por `tconst`, así que esas filas son estables entre versiones del dataset y producen un `datos.dat` de unos 27 MB, por encima de los 20 MB exigidos. La clave es el número de `tconst`: `tt0111161` → `111161`.

`ArchivoDatos`

`datos.dat` guarda los registros uno detrás de otro, con longitud variable:

`long clave` | `int largo` | `largo bytes: la línea TSV completa en UTF-8, sin el salto de línea`

La **posición** de un registro es el byte donde empieza su clave; es lo que guarda el índice.

Método	Funcionalidad
`static ArchivoDatos crear(String ruta)` / `abrir(String ruta)`	Para escribir (construcción) o para leer.
`long agregar(long clave, String linea)`	Agrega al final y retorna la posición.
`String leer(long pos)`	Lee el registro de esa posición: un *seek* y una lectura.
`long busquedaSecuencial(long clave)`	Recorre el archivo desde el principio; posición o -1.
`long registrosLeidos()`, `reiniciarContadores()`, `long tamano()`, `close()`	Contador de registros leídos por `leer` y por la búsqueda secuencial.

`BMasApp`: cada comando es una ejecución separada

```
java BMasApp construir title.basics.tsv [filas=300000] [tamPagina=4096] [--aleatorio]
java BMasApp buscar tt0111161        # o el número: 111161
java BMasApp rango 100000 100999 [mostrar=10]
java BMasApp secuencial 111161
java BMasApp info                    # P, M, L, raiz, niveles, paginas, ocupacion
java BMasApp verificar
java BMasApp pagina 345
```

`construir` genera `datos.dat` e `indice.dat`. Todos los demás comandos trabajan **solo** con esos dos archivos; ninguno vuelve a leer el TSV. `buscar` imprime el registro y las páginas leídas con su camino, por ejemplo:

```
paginas leidas: 3 (#345 -> #861 -> #854)
pos=9842010  tt0111161  movie  The Shawshank Redemption  ...
```

Con `-aleatorio`, `construir` escribe `datos.dat` en el orden del archivo, guarda las claves y posiciones en dos arreglos `long[]`, los baraja con el Fisher–Yates de abajo (semilla 2026) y recién entonces inserta en el índice.

```
Random rng = new Random(2026);
for (int i = n - 1; i > 0; i--) {
    int j = rng.nextInt(i + 1);
    // intercambiar a[i] con a[j] (y, si hay arreglos paralelos, tambien b[i] con b[j])
}
```

`ExperimentoBMas`: cuatro tablas que imprime el programa

No se transcriben a mano: el programa las imprime (por ejemplo con comandos `sintetico`, `imdb` y `busquedas`). Todas las mediciones, sin buffer de páginas.

Tabla A — estructura (sintética, determinista). Claves $1..N$ con $N = 1\,000\,000$ y posición = clave, para $P \in \{128, 256, 512, 1024, 2048, 4096\}$, construyendo dos árboles: inserción en orden creciente, e inserción de la permutación de Fisher–Yates con semilla 2026. Columnas: `P`, `M`, `L`, y para cada orden `niveles`, `hojas`, `ocupacionHojas`, `ocupacionInternos`. Verifiquen las invariantes de cada árbol. Esta tabla tiene que coincidir con la del docente (el millón de claves en 12 árboles sin buffer toma unos minutos en disco; verifiquen primero con N menor).

Tabla B — construcción con IMDb. Para la construcción en orden del archivo y con `-aleatorio`: registros, tamaño de `datos.dat` y de `indice.dat`, P, M, L, niveles, páginas (hojas e internos), ocupación de hojas y de internos, páginas escritas y tiempo.

Tabla C — búsquedas. Sobre el índice construido en orden del archivo:

1. Obtengan todas las claves con `buscarRango(Long.MIN_VALUE, Long.MAX_VALUE, ...)` en un `long[]` de tamaño `cantidadClaves()` (son 8 bytes por clave, no el archivo; está permitido para el experimento).
2. Con `Random rng = new Random(2026)`: 1000 claves **existentes** `claves[rng.nextInt(n)]`; luego, con el mismo `rng`, 1000 **inexistentes** `x = min + (long)(rng.nextDouble()*(max-min+1))`, descartando las que estén en el arreglo (búsqueda binaria propia).
3. Midan: B+ con las 1000 existentes (incluyendo leer el registro de `datos.dat`), B+ con las 1000 inexistentes, búsqueda secuencial con las primeras 50 existentes y con las primeras 50 inexistentes.

Columnas: método, consultas, páginas de índice por consulta, registros leídos por consulta, microsegundos por consulta.

Tabla D — rangos. Siguiendo con el mismo `rng`: para cada ancho $w \in \{10, 100, 1000, 10\,000\}$, 100 rangos $[x, x + w - 1]$ con `x = claves[rng.nextInt(n)]`. Columnas: w, registros entregados por consulta, páginas leídas por consulta, hojas leídas por consulta (= páginas − (niveles −1)), microsegundos por consulta.

Por qué contamos páginas y no solo milisegundos. El tiempo depende de la máquina y, sobre todo, del cache del sistema operativo: con 27 MB, después de la primera pasada el archivo entero queda en RAM y el disco casi no se toca. Las páginas leídas dependen **solo del algoritmo** y son las que tendría que traer un disco frío. Reporten los dos, y razonen con las páginas.

Análisis (va al README y se defiende)

En el `README.md` (máximo tres páginas, más las tablas), justificando cada punto:

1. **Capacidad de página.** Deduzcan L y M en función de P para este formato; valores y bytes sin usar para $P = 60$ y $P = 4096$; por qué $M \neq L$.
2. **Altura.** Acoten los niveles de un B+ con N claves: el mínimo, con todas las páginas llenas, y el máximo, con todas en su ocupación mínima (I3). Comparen con las columnas `niveles` de las Tablas A y B, y con el ABB de la figura de la motivación.

3. **E/S por operación.** `buscar`: `niveles()` lecturas, en todo caso; `insertar`: `niveles()` lecturas y $2s + 1$ escrituras. Concluyan $\Theta(\log_M N)$ accesos a disco y $\Theta(\log N)$ comparaciones (búsqueda binaria dentro de cada página).

4. **Ocupación.** Expliquen por qué la inserción en orden creciente deja las hojas cerca del 50 % y **predigan exactamente** la columna `hojas` creciente de la Tabla A a partir de N y L. Comparen con la inserción aleatoria y digan qué consecuencia tiene en el tamaño del índice y en los niveles (miren las filas $P = 128$ y $P = 2048$). ¿En qué orden llegan las claves de IMDb?

5. **Secuencial contra índice.** Con la Tabla C: registros leídos por la búsqueda secuencial en una clave existente y en una inexistente, contra páginas del B+. ¿Qué crecimiento tiene cada uno en función de N?

6. **Rango.** Expresen las páginas de un rango en función de los niveles, de la cantidad s de registros entregados y de la ocupación de las hojas; contrasten con la Tabla D y con lo que costarían s búsquedas independientes.

Cuando se pide la cota «lo más ajustada posible», la respuesta esperada es Θ, no O.

Uso de inteligencia artificial (va al README, obligatorio)

La guía de la asignatura permite usar plataformas de inteligencia artificial en los TP, con una condición: **especificar los prompts utilizados y los cambios introducidos**. Una respuesta de la IA entregada tal cual se considera plagio. Por eso el `README.md` debe terminar con una sección **«Uso de IA»** que contenga:

1. **Qué hicieron con IA y qué hicieron manualmente**, parte por parte. Como mínimo: formato de página y cálculo de M y L; `buscar`; split de hoja; split de interno y raíz nueva; `buscarRango`; persistencia (`crear`/`abrir`/`close`); `verificarInvariantes`; `TestArbolBMas`; `ArchivoDatos` y `BMasApp`; `ExperimentoBMas`; análisis del README. Para cada parte indiquen si fue *manual*, *con ayuda de IA* o *generada por IA y modificada*.
2. **Herramienta y prompts**: qué herramienta usaron (ChatGPT, Copilot, Gemini, Claude, etc.) y los prompts relevantes, copiados textualmente.
3. **Cambios introducidos**: qué corrigieron o reescribieron de lo que propuso la IA y por qué (errores encontrados, reglas del enunciado que no respetaba, colecciones prohibidas, etc.).

Si no usaron IA en ninguna parte, lo declaran en esa sección. La declaración no resta puntos por sí misma: lo que se evalúa es que el código sea suyo y que lo puedan explicar. **Lo declarado se contrasta en la defensa**: cualquier parte marcada como hecha con IA puede ser la primera que se les pida explicar o modificar. Un README sin esta sección se considera incompleto (ítem de análisis del Ejercicio 2), y una declaración que no coincide con lo que se observa en la defensa se trata como falta a la Declaración de Honor.

Rúbrica — Ejercicio 2

Rúbrica	Puntaje
`ArchivoDatos` con el formato de registro, posiciones, lectura aleatoria y búsqueda secuencial	4
`BMasApp`: construcción con ≥ 20 MB y consultas en ejecuciones separadas, sin releer el TSV	3
`ExperimentoBMas`: las Tablas A, B, C y D impresas por el programa según la especificación	7
Análisis en el README: capacidad, altura, E/S, ocupación, secuencial contra índice, rangos, relación con **sus** tablas; sección «Uso de IA» completa	10
Orden, estilo, comentarios y datos	1
Total	**25**

Defensa oral (25p)

Se realiza en clase los días **14 y 15 de octubre**. Cada grupo dispone de **15 minutos**. La defensa es **individual dentro del grupo**: los docentes eligen a quién le preguntan cada punto. Es una prueba práctica sobre el código entregado, no una presentación.

La defensa es condición necesaria del TP completo

El puntaje de los Ejercicios 1 y 2 solo se acredita a cada integrante si ese integrante demuestra en la defensa que comprende el código entregado. Quien no pueda explicar el trabajo de su grupo recibe **0 en los 100 puntos del TP**, sin afectar a los demás integrantes.

Condiciones

- El código a defender es el entregado el 12/10. No se aceptan modificaciones posteriores.
- Se debe traer el proyecto compilando y listo para ejecutar, con el TSV descargado y `datos.dat`/`indice.dat` ya construidos.
- **Todos los integrantes deben estar presentes.** Un integrante ausente sin justificación recibe 0 en el TP.
- Los docentes pueden pedir ejecutar el código con entradas, tamaños de página y claves distintas a las del enunciado, y cerrar y volver a abrir el programa en cualquier momento.

Rúbrica — Defensa

Rúbrica	Puntaje
Explica la representación de una página, cómo se obtiene su posición en `indice.dat` y de dónde salen M y L, sin leer el código	5
Traza en vivo búsquedas e inserciones: qué páginas se leen, por qué se divide un nodo, qué se copia y qué sube, qué páginas se asignan	7
Demuestra y explica la persistencia: qué guarda la cabecera y cómo se reabre el árbol sin reconstruirlo	3
Relaciona altura, tamaño de página, *fan-out*, cantidad de claves y accesos a disco, con sus tablas	5
Responde a una modificación propuesta en el momento describiendo el cambio necesario	3
Dominio individual: cada integrante responde por cualquier parte del código entregado, incluidas las que declaró hechas con IA	2
Total	**25**

Criterio de aplicación

Situación	Efecto
El integrante explica con fluidez y sin leer el código	Puntaje completo del ítem
Explica leyendo el código, pero con comprensión	Dos tercios del ítem
Sabe qué hace el código pero no por qué se diseñó así	Un tercio del ítem
No puede explicar código que su grupo entregó	**0 en el TP completo** para ese integrante
Se detecta que un integrante no participó del desarrollo	**0 en el TP completo** para ese integrante

Entrega

- **Formato:** un único archivo `g##-TP3.zip` con todos los `.java` y un `README.md` con las decisiones de diseño, las cuatro tablas, el análisis pedido y la sección «Uso de IA». **Sin** el TSV ni los `.dat`.
- **Fecha límite:** lunes 12/10/2026, 23:59 hs, hora del servidor de Educa.
- **Vía:** exclusivamente la plataforma Educa, en la actividad correspondiente del aula virtual. No se reciben trabajos por correo.
- No se reciben entregas pasada la fecha límite (Planeamiento de Cátedra 2026-2S).